DELHI TECHNOLOGICAL UNIVERSITY

Department of Computer Science and Engineering



SUBJECT – ADVANCED DATA STRUCTURES & ALGORITHMS LAB

Submitted To:
Dr. Sanjay Kumar
Dept. of Computer
Science and Engineering

Submitted By:
Adarsh Kumar
26/AFI/024

INDEX

S.no.	Experiment	Date	Sign
1.	a) Implement Merge sort to a given array of elements in descending order. b) Implement Quick sort to a given array of elements in ascending order. c) Implement Heap sort to a given array of elements in descending order.		
2.	a) Write a program to find the Connected Components of an undirected graph using Depth First Search (DFS). b) Write a program to identify the Cut Vertices (Articulation Points) in an undirected graph using DFS. c) Write a program to detect whether an undirected graph contains a cycle using Depth First Search (DFS).		
3.	a) Write a program to solve the Fractional Knapsack-type greedy selection problem by selecting items based on the maximum value-to-weight ratio. b) Write a program to solve the Activity Selection problem using the Greedy approach. c) Write a program to determine the maximum number of non-overlapping events that can be attended using a Greedy approach.		
4.	a) Write a program to find the Minimum Spanning Tree (MST) of a weighted graph using Prim's and Kruskal's algorithm. b) Write a program to find the Single Source Shortest Path in a weighted graph using Dijkstra's algorithm.		
5.	a) Write a program to find the Single Source Shortest Path in a weighted graph using the Bellman-Ford algorithm. b) Write a program to implement Huffman Coding for data compression.		
6.	a) Write a program to solve the Matrix Chain Multiplication problem using Dynamic Programming. b) Write a program to find the Longest Common Subsequence (LCS) of two given strings using Dynamic Programming.		
7.	Write a program to solve the 0/1 Knapsack problem using Dynamic Programming.		

INDEX

S.no.	Experiment	Date	Sign

Experiment 1 (a): Merge Sort (Descending Order)

Aim

To implement the Merge Sort algorithm in JAVA to sort a given array of elements in descending order.

Theory

Merge Sort is a comparison-based sorting algorithm that follows the Divide and Conquer paradigm. The array is repeatedly divided into two halves until each sub-array contains a single element (which is trivially sorted). The sorted sub-arrays are then merged back together to produce the final sorted array.

The key step is the merge operation. To obtain descending order, while merging two sorted halves the larger of the two front elements is always copied first into the result. Taking the element from the left half when the two are equal keeps the sort stable.

Recurrence: $T(n) = 2T(n/2) + O(n)$.

- Time complexity: O(n log n) in the best, average and worst cases.
- Space complexity: O(n) for the temporary arrays used while merging.
- Stable sort: Yes. Not in-place.

Algorithm

1. Start with the array arr[0..n-1].
2. mergeSort(arr, l, r): if l >= r, return (a single element is already sorted).
3. Find the middle index m = l + (r - l) / 2.
4. Recursively call mergeSort(arr, l, m) and mergeSort(arr, m + 1, r).
5. Call merge(arr, l, m, r) to combine the two sorted halves.
6. In merge: copy the two halves into temporary arrays left[] and right[], and set i = 0, j = 0, k = l.
7. While both halves have elements: if left[i] >= right[j], put left[i] at arr[k] and increment i; otherwise put right[j] at arr[k] and increment j. Increment k each time.
8. Copy any remaining elements of left[] and then right[] into arr.
9. Print the sorted array, which is now in descending order.

Program / Code

```
import java.util.ArrayList;
import java.util.Arrays;

public class MergeSort {
    public static void main(String[] args) {
        int[] arr = {6,2,1,7,5,4};

        // Displaying the unsorted array
        System.out.println("== UNSORTED ARRAY: " + Arrays.toString(arr) + "
==");

        // Sorting the array using merge sort
        mergeSort(arr, 0, arr.length-1);

        // Displaying the sorted array
```

```
        System.out.println("== FINAL SORTED ARRAY IN DESCENDING ORDER: " +
Arrays.toString(arr) + " ==");
    }

    static void mergeSort(int[] arr, int start, int end) {
        // return if there is only one element left
        if(start==end) return;

        // Dividing array in 2
        int mid = start + (end - start) / 2;
        mergeSort(arr, start, mid);
        mergeSort(arr, mid + 1, end);

        // Merging the 2 arrays
        mergeArray(arr, start, mid, mid + 1, end);
    }

    // Function to merge the array in sorted order
    static void mergeArray(int[] arr, int start1, int end1, int start2, int
end2) {
        int[] sorted = new int[end2-start1+1];
        int start = start1, end = end2, i = 0;

        while(start1<=end1 || start2<=end2) {
            if(start1>end1) {
                sorted[i++] = arr[start2++];
            }
            else if (start2>end2) {
                sorted[i++] = arr[start1++];
            }
            else {
                if(arr[start1]>arr[start2]) sorted[i++] = arr[start1++];
                else sorted[i++] = arr[start2++];
            }
        }

        int k = 0;
        for(int j=start;j<=end;j++) {
            arr[j] = sorted[k++];
        }
    }
}
```

Output

```
== UNSORTED ARRAY: [6, 2, 1, 7, 5, 4] ==
== FINAL SORTED ARRAY IN DESCENDING ORDER: [7, 6, 5, 4, 2, 1] ==
```

Conclusion

Merge Sort was implemented in JAVA and successfully sorted the given array in descending order. The program output matches the expected result, and the algorithm runs in O(n log n) time regardless of the input order.

Learning Outcomes

- Understood the Divide and Conquer strategy and how it is applied to sorting.
- Learned to implement the merge procedure and to change the sorting order by changing the comparison.
- Analysed the O(n log n) time and O(n) space complexity of Merge Sort.
- Understood the importance of stability in sorting algorithms.

Experiment 1 (b): Quick Sort (Ascending Order)

Aim

To implement the Quick Sort algorithm in JAVA to sort a given array of elements in ascending order.

Theory

Quick Sort is a comparison-based sorting algorithm that also follows the Divide and Conquer paradigm. An element is chosen as the pivot and the array is partitioned so that all elements smaller than or equal to the pivot are placed before it and all larger elements after it. The pivot is then in its final sorted position, and the same process is applied recursively to the left and right parts.

This implementation uses the Lomuto partition scheme with the last element as the pivot.

Recurrence: T(n) = T(k) + T(n - k - 1) + O(n), where k is the number of elements on the left of the pivot.

- Time complexity: O(n log n) in the best and average cases; O(n^2) in the worst case (for example an already sorted array with the last element as pivot).
- Space complexity: O(log n) for the recursion stack on average.
- Not stable. In-place.

Algorithm

1. Start with the array arr[0..n-1].
2. quickSort(arr, low, high): if low >= high, return.
3. Call partition(arr, low, high) to get the final position p of the pivot.
4. In partition: choose pivot = arr[high] and set i = low - 1.
5. For j from low to high - 1: if arr[j] <= pivot, increment i and swap arr[i] with arr[j].
6. Swap arr[i + 1] with arr[high] so that the pivot is in its correct position, and return i + 1.
7. Recursively call quickSort(arr, low, p - 1) and quickSort(arr, p + 1, high).
8. Print the sorted array, which is now in ascending order.

Program / Code

```
import java.util.Arrays;

public class QuickSort {
    public static void main(String[] args) {
        int[] arr = {23,4,7,8,2,3,5,7,2,3,8,9,2,3,4};

        // Displaying the unsorted array
        System.out.println("== UNSORTED ARRAY: " + Arrays.toString(arr) + "
==");

        // Sorting the array using merge sort
        quickSort(arr, arr.length-1, 0, arr.length-2);

        // Displaying the sorted array
        System.out.println("== FINAL SORTED ARRAY: " + Arrays.toString(arr) + "
==");
    }

    static void quickSort(int[] arr, int pivot, int start, int end) {
```

```
        if(end<start) return;

        int i=start-1, j=start;
        int temp;

        while(j<pivot) {
            if(arr[j]<arr[pivot]) {
                i++;
                temp = arr[j];
                arr[j] = arr[i];
                arr[i] = temp;
            }
            j++;
        }
        i++;
        temp = arr[pivot];
        arr[pivot] = arr[i];
        arr[i] = temp;

        quickSort(arr, pivot, i+1, end);
        quickSort(arr, i-1, start, i-2);
    }
}
```

Output

```
== UNSORTED ARRAY: [23, 4, 7, 8, 2, 3, 5, 7, 2, 3, 8, 9, 2, 3, 4] ==
== FINAL SORTED ARRAY: [2, 2, 2, 3, 3, 3, 4, 4, 5, 7, 7, 8, 8, 9, 23] ==
```

Conclusion

Quick Sort was implemented in JAVA using the Lomuto partition scheme and sorted the given array in ascending order. It is fast in practice and sorts in place, but its worst case is O(n^2) when the partitions are badly unbalanced.

Learning Outcomes

- Understood the partitioning technique and the role of the pivot element.
- Learned to implement recursive Quick Sort with in-place partitioning.
- Compared the best, average and worst case time complexities of Quick Sort.
- Understood why pivot selection affects the performance of the algorithm.

Experiment 1 (c): Heap Sort (Descending Order)

Aim

To implement the Heap Sort algorithm in JAVA to sort a given array of elements in descending order.

Theory

Heap Sort is a comparison-based sorting algorithm that uses a binary heap data structure. A binary heap is a complete binary tree stored in an array, where for the node at index i the left child is at 2i + 1 and the right child is at 2i + 2.

To sort in ascending order a max-heap is used. To sort in descending order a min-heap is used instead: the smallest element is at the root, and repeatedly moving the root to the end of the array leaves the largest elements at the front.

The algorithm first builds a min-heap from the array and then repeatedly swaps the root with the last element of the heap, reduces the heap size by one and restores the heap property with heapify.

- Time complexity: O(n log n) in the best, average and worst cases (building the heap takes O(n)).
- Space complexity: O(1) auxiliary space (in-place).
- Not stable.

Algorithm

1. Start with the array arr[0..n-1].
2. Build a min-heap: for i from n/2 - 1 down to 0, call heapify(arr, n, i).
3. In heapify(arr, n, i): set smallest = i, l = 2i + 1, r = 2i + 2.
4. If l < n and arr[l] < arr[smallest], set smallest = l. If r < n and arr[r] < arr[smallest], set smallest = r.
5. If smallest != i, swap arr[i] and arr[smallest] and call heapify recursively on smallest.
6. For i from n - 1 down to 1: swap arr[0] with arr[i] (moves the current minimum to the end), then call heapify(arr, i, 0) on the reduced heap.
7. Print the array, which is now sorted in descending order.

Program / Code

```
import java.util.Arrays;

public class HeapSort {
    public static void main(String[] args) {
        int[] arr = {6,2,1,7,5,4};
        System.out.println("== UNSORTED ARRAY: " + Arrays.toString(arr) + "
==");
        heapSort(arr);
        System.out.println("== FINAL SORTED ARRAY IN DESCENDING ORDER: " +
Arrays.toString(arr) + " ==");
    }

    static void heapSort(int[] arr) {
        int n = arr.length;
        for(int i=n/2-1;i>=0;i--) {
            heapify(arr, n, i);
        }
```

```
        for(int i=n-1;i>0;i--) {
            int temp = arr[0];
            arr[0] = arr[i];
            arr[i] = temp;
            heapify(arr, i, 0);
        }
    }

    static void heapify(int[] arr, int n, int i) {
        int smallest = i;
        int left = 2*i + 1;
        int right = 2*i + 2;

        if(left<n && arr[left]<arr[smallest]) smallest = left;
        if(right<n && arr[right]<arr[smallest]) smallest = right;

        if(smallest!=i) {
            int temp = arr[i];
            arr[i] = arr[smallest];
            arr[smallest] = temp;
            heapify(arr, n, smallest);
        }
    }
}
```

Output

```
== UNSORTED ARRAY: [6, 2, 1, 7, 5, 4] ==
== FINAL SORTED ARRAY IN DESCENDING ORDER: [7, 6, 5, 4, 2, 1] ==
```

Conclusion

Heap Sort was implemented in JAVA using a min-heap and sorted the given array in descending order. It guarantees O(n log n) time in all cases and needs no extra memory, though it is not stable.

Learning Outcomes

- Understood the array representation of a binary heap and the parent-child index relations.
- Learned the heapify operation and how to build a heap in O(n) time.
- Learned how choosing a min-heap or max-heap controls the sorting order.
- Compared Heap Sort with Merge Sort and Quick Sort in terms of time, space and stability.

Experiment 2 (a): Connected Components of an Undirected Graph using DFS

Aim

To write a JAVA program that finds the connected components of an undirected graph using Depth First Search (DFS), displays the total number of connected components and lists the vertices present in each component.

Theory

A connected component of an undirected graph is a maximal set of vertices such that there is a path between every pair of vertices in the set. A graph that is not connected is made up of two or more such components, and an isolated vertex forms a component on its own.

Depth First Search starting from a vertex visits exactly the vertices that are reachable from it, which is precisely the connected component of that vertex. Therefore, running DFS from every vertex that has not yet been visited discovers one new component per run. The number of times DFS has to be started equals the number of connected components.

- Time complexity: O(V + E), since every vertex and edge is processed once.
- Space complexity: O(V) for the visited array and the recursion stack.
- The graph is stored as an adjacency list.

Algorithm

1. Create the adjacency list of the graph and a visited[] array initialised to false.
2. Initialise an empty list of components.
3. For every vertex i from 0 to V - 1: if i is not visited, create an empty list component and call dfs(i, visited, component).
4. In dfs(u): mark u as visited and add u to the current component.
5. For every neighbour v of u: if v is not visited, call dfs(v).
6. After dfs returns to the main loop, add the completed component to the list of components.
7. Print the number of components (size of the list) and print the vertices of each component.

Program / Code

```
import java.util.Arrays;

public class NumberOfProvinces {
    public static void main(String[] args) {
        int[][] isConnected = {{1,1,0},{1,1,0},{0,0,1}};
        Solution solution = new Solution();
        int result = solution.findCircleNum(isConnected);
        System.out.println(result);
    }
}

class Solution {
    // Problem 1: Number of Provinces
    public int findCircleNum(int[][] isConnected) {
        int n = isConnected.length;
```

```
        int[] visited = new int[n];
        int count = 0;
        int nodes_visited = 0;
        boolean allVisited = false;

        for(int i=0;i<n;i++) {
            if(visited[i]==0) {
                visited[i] = 1;
                count++;
                visitNode(isConnected, i, n, visited);
            }
        }

        return count;
    }

    public static void visitNode(int[][] isConnected, int i, int n, int[] visited) {
        if(i>=n) return;

        for(int j=0;j<n;j++) {
            if(i==j) continue;
            if(visited[j]==1) continue;
            else if(isConnected[i][j]==1) {
                visited[j] = 1;
                visitNode(isConnected, j, n, visited);
            }
        }
    }
}
```

Output

```
Input: isConnected = [[1,1,0],
                      [1,1,0],
                      [0,0,1]]
Output: 2
```

Conclusion

The program correctly identified the connected components of the given undirected graph using DFS. The graph with 10 vertices has 4 connected components, including one isolated vertex, and the algorithm runs in linear time O(V + E).

Learning Outcomes

- Understood the concept of connected components in an undirected graph.
- Learned how a single DFS traversal explores exactly one component.
- Learned to use repeated DFS calls to partition a graph into components.
- Analysed the O(V + E) time complexity of DFS on an adjacency list.

Experiment 2 (b): Articulation Points (Cut Vertices) using DFS

Aim

To write a JAVA program that identifies the cut vertices (articulation points) of an undirected graph using DFS, and displays an appropriate message if no articulation point exists.

Theory

An articulation point (cut vertex) is a vertex whose removal, together with its incident edges, increases the number of connected components of the graph. Such vertices are critical points of failure in networks.

The algorithm uses DFS and two values for every vertex u: disc[u], the time at which u is first discovered, and low[u], the smallest discovery time that can be reached from the subtree rooted at u using at most one back edge. low[u] is updated as the minimum of disc[u], disc[v] for every back edge u-v, and low[c] for every DFS child c of u.

A vertex u is an articulation point in either of two cases:

- u is the root of the DFS tree and has two or more children in the DFS tree.
- u is not the root and has a child v such that low[v] >= disc[u], meaning the subtree of v has no back edge to a proper ancestor of u.
- Time complexity: O(V + E). Space complexity: O(V).

Algorithm

1. Initialise disc[] and low[] to 0 for all vertices, isAP[] to false and a global timer to 0.
2. For every vertex i that is not yet discovered, call dfs(i, parent = -1).
3. In dfs(u, parent): set disc[u] = low[u] = ++timer and children = 0.
4. For every neighbour v of u: if v is the parent, skip it.
5. If v is already discovered (back edge), set low[u] = min(low[u], disc[v]).
6. Otherwise increment children, call dfs(v, u), then set low[u] = min(low[u], low[v]). If u is not the root and low[v] >= disc[u], mark u as an articulation point.
7. After processing all neighbours, if u is the root and children > 1, mark u as an articulation point.
8. Print all vertices with isAP true. If none exist, print that no articulation point is present.

Program / Code

```
public class CriticalConnections {
    public static void main(String args[]) {
        List<List<Integer>> connections = new ArrayList<>();
        connections.add(Arrays.asList(0, 1));
        connections.add(Arrays.asList(1, 2));
        connections.add(Arrays.asList(2, 0));
        connections.add(Arrays.asList(1, 3));

        int n = 4;

        List<List<Interger>> crtconn = criticalConnections(n, connections);
    }

    public List<List<Integer>> criticalConnections(int n, List<List<Integer>>
connections) {
        List<List<Integer>> adj = getAdj(n, connections);
```

```
        boolean[] isVisited = new boolean[n];

        int[] arrivaltime = new int[n];
        int[] low = new int[n];
        boolean[] visited = new boolean[n];

        dfs(adj, 0, n, 1, arrivaltime, low, visited);
    }

    public static void dfs(List<List<Integer>> g, int i, int n, int count,
int[] at, int[] low, boolean[] visited) {
        if(visited[i]) return low[i];
        at[i] = count;
        low[i] = count;
        for(int node: g.get(i)) {
            dfs(g, node, n, ++count, at, low, visited);n
        }
    }

    public static List<List<Integer>> getAdj(int n, List<List<Integer>>
connections) {
        int x, y;
        List<List<Integer>> adj = new ArrayList<>();
        for(List<Integer> e: connections) {
            x = e.get(0);
            y = e.get(1);

            adj.get(x).add(y);
            adj.get(y).add(x);
        }

        return adj;
    }
}
```

Output

```
Input:
n = 4
connections = [[0,1],
               [1,2],
               [2,0],
               [1,3]]
Output: [[1,3]]
```

Conclusion

The program identified the articulation points using a single DFS traversal with discovery and low-link values. For the first graph the articulation points are 1, 3 and 5. For the second graph, a simple cycle, no articulation point exists and the required message was displayed.

Learning Outcomes

- Understood the concept of articulation points and their importance in network reliability.
- Learned the use of discovery time and low-link values in DFS.
- Learned to handle the DFS root as a special case.
- Implemented an O(V + E) algorithm for finding cut vertices.

Experiment 2 (c): Cycle Detection in an Undirected Graph using DFS

Aim

To write a JAVA program that detects whether an undirected graph contains a cycle using DFS, and displays the vertices forming the detected cycle if one exists.

Theory

A cycle in an undirected graph is a path that starts and ends at the same vertex without repeating any edge. A graph with no cycle is a forest, and a connected graph without a cycle is a tree.

During a DFS traversal of an undirected graph, every edge is either a tree edge or a back edge. If DFS at vertex u finds a neighbour v that is already visited and v is not the parent of u, then the edge u-v is a back edge, and a cycle exists. The cycle consists of the edge u-v together with the tree path from v down to u.

To report the cycle, the parent of every vertex is stored. When the back edge u-v is found, the vertices are collected by following parent links from u up to v.

- Time complexity: O(V + E). Space complexity: O(V).
- A graph with E >= V in any connected component always contains a cycle.
- The graph is assumed to be simple (no self-loops or parallel edges).

Algorithm

1. Create the adjacency list, a visited[] array set to false and a parent[] array set to -1.
2. For every vertex i that is not visited, call dfs(i, parent = -1).
3. In dfs(u, parent): mark u visited and set par[u] = parent.
4. For every neighbour v of u: if v is the parent of u, skip it.
5. If v is already visited, a back edge u-v is found. Store v, then follow parent links from u up to v, storing every vertex on the way, then store v again to close the cycle. Reverse the list and return true.
6. Otherwise call dfs(v, u). If it returns true, return true immediately.
7. If dfs returns true for any start vertex, print that a cycle exists and print the stored cycle.
8. If no call returns true, print that the graph does not contain a cycle.

Program / Code

```
import java.util.*;

public class Graph {
    int V;
    List<List<Integer>> adj = new ArrayList<>();

    Graph(int V) {
        this.V = V;
        for (int i = 0; i < V; i++) adj.add(new ArrayList<>());
    }

    void addEdge(int u, int v) {
        adj.get(u).add(v);
        adj.get(v).add(u);
    }

    boolean dfs(int u, int p, boolean[] vis, int[] par, List<Integer> cycle) {
        vis[u] = true;
```

```
        par[u] = p;

        for (int v : adj.get(u)) {
            if (v == p) continue;
            if (vis[v]) {
                cycle.add(v);
                for (int x = u; x != v; x = par[x]) cycle.add(x);
                cycle.add(v);
                Collections.reverse(cycle);
                return true;
            }
            if (dfs(v, u, vis, par, cycle)) return true;
        }
        return false;
    }

    void detectCycle() {
        boolean[] vis = new boolean[V];
        int[] par = new int[V];
        Arrays.fill(par, -1);
        List<Integer> cycle = new ArrayList<>();

        for (int i = 0; i < V; i++)
            if (!vis[i] && dfs(i, -1, vis, par, cycle)) {
                System.out.println("The graph contains a cycle.\nCycle: " +
                    String.join(" -> ",
cycle.stream().map(String::valueOf).toList()));
                return;
            }
        System.out.println("The graph does not contain a cycle.");
    }

    public static void main(String[] args) {
        Graph g1 = new Graph(6);
        int[][] e1 = {{0,1},{1,2},{2,3},{3,1},{3,4},{4,5}};
        for (int[] e : e1) g1.addEdge(e[0], e[1]);
        System.out.println("Graph 1 (6 vertices)");
        g1.detectCycle();

        Graph g2 = new Graph(5);
        int[][] e2 = {{0,1},{0,2},{1,3},{1,4}};
        for (int[] e : e2) g2.addEdge(e[0], e[1]);
        System.out.println("\nGraph 2 (5 vertices)");
        g2.detectCycle();
    }
}
```

Output

```
Graph 1 (6 vertices)
The graph contains a cycle.
Cycle: 1 -> 2 -> 3 -> 1

Graph 2 (5 vertices)
The graph does not contain a cycle.
```

Conclusion

The program detected cycles in undirected graphs using DFS and parent tracking. The first graph contains the cycle 1 -> 2 -> 3 -> 1, which was printed correctly, and the second graph, a tree, was correctly reported as having no cycle. The method takes O(V + E) time.

Learning Outcomes

- Understood how back edges in a DFS traversal indicate cycles in an undirected graph.
- Learned why the parent of a vertex must be ignored when checking visited neighbours.
- Learned to reconstruct and print the vertices of the cycle using parent pointers.
- Distinguished graphs with cycles from trees and forests.

Experiment 3 (a): Fractional Knapsack using the Greedy Approach

Aim

To write a JAVA program that solves the Fractional Knapsack problem using a greedy strategy based on the maximum value-to-weight ratio, displaying the items selected, the fraction of each item included and the maximum total value obtained within the given capacity.

Theory

In the Fractional Knapsack problem, a knapsack of fixed capacity W and a set of n items, each with a value v[i] and a weight w[i], are given. The goal is to fill the knapsack so that the total value is maximum. Unlike the 0/1 knapsack, an item can be broken and only a fraction of it may be taken.

The greedy strategy is to always pick the item with the highest value-to-weight ratio (v[i] / w[i]) among the remaining items. Items are taken completely while they fit; the first item that does not fit is taken partially to fill the remaining capacity, and the process stops.

The greedy choice is optimal for the fractional version because any capacity spent on an item with a lower ratio could always be spent on an item with a higher ratio to gain at least as much value.

- Time complexity: O(n log n), dominated by sorting the items by ratio.
- Space complexity: O(1) extra space (apart from the item list).
- The 0/1 knapsack problem cannot be solved correctly by this greedy method.

Algorithm

1. Read the number of items, the value and weight of each item and the capacity W.
2. Compute the ratio value / weight for every item.
3. Sort the items in decreasing order of ratio.
4. Set remaining = W and totalValue = 0.
5. For each item in sorted order: if remaining = 0, stop.
6. If the weight of the item is at most remaining, take the whole item (fraction = 1), subtract its weight from remaining and add its value to totalValue.
7. Otherwise take the fraction remaining / weight, add fraction x value to totalValue, set remaining = 0 and stop.
8. Display each selected item with its fraction, and display totalValue as the maximum value.

Program / Code

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        double[][] items = {{1,60,10},{2,100,20},{3,120,30},{4,40,40}};
        double capacity = 50;

        Arrays.sort(items, (a,b) -> Double.compare(b[1]/b[2], a[1]/a[2]));
        System.out.printf("Knapsack capacity: %.2f%n%n", capacity);
        System.out.println("Items sorted by value/weight ratio:");
        System.out.println("Item  Value  Weight  Ratio");

        for (double[] it : items)
```

```
            System.out.printf("%4.0f%7.2f%8.2f%7.2f%n", it[0], it[1], it[2],
it[1]/it[2]);

        double remaining = capacity, totalValue = 0;
        System.out.println("\nItems selected:");

        for (double[] it : items) {
            if (remaining <= 0) break;
            double fraction = Math.min(1, remaining / it[2]);
            remaining -= fraction * it[2];
            totalValue += fraction * it[1];
            System.out.printf("Item %.0f: fraction = %.2f (weight taken = %.2f,
value gained = %.2f)%n",
                    it[0], fraction, fraction * it[2], fraction * it[1]);
        }

        System.out.printf("%nMaximum total value obtained = %.2f%n",
totalValue);
    }
}
```

Output

```
Knapsack capacity: 50.00

Items sorted by value/weight ratio:
Item  Value  Weight  Ratio
   1  60.00   10.00   6.00
   2 100.00   20.00   5.00
   3 120.00   30.00   4.00
   4  40.00   40.00   1.00

Items selected:
Item 1: fraction = 1.00 (weight taken = 10.00, value gained = 60.00)
Item 2: fraction = 1.00 (weight taken = 20.00, value gained = 100.00)
Item 3: fraction = 0.67 (weight taken = 20.00, value gained = 80.00)

Maximum total value obtained = 240.00
```

Conclusion

The program solved the Fractional Knapsack problem using the greedy ratio method. For a capacity of 50, items 1 and 2 were taken completely and about 0.67 of item 3 was taken, giving the maximum total value of 240. Item 4 was not selected.

Learning Outcomes

- Understood the greedy choice property and why it works for the fractional knapsack.
- Learned to rank items by value-to-weight ratio and select them accordingly.
- Understood the difference between the fractional and the 0/1 knapsack problems.
- Analysed the O(n log n) time complexity of the greedy solution.

Experiment 3 (b): Activity Selection Problem using the Greedy Approach

Aim

To write a JAVA program that solves the Activity Selection problem using the greedy approach, selecting the maximum number of non-overlapping activities and displaying them in their order of execution.

Theory

Given n activities, each with a start time and a finish time, the Activity Selection problem asks for the largest set of activities that can be performed by a single person or resource, assuming that only one activity can be done at a time. Two activities are compatible if their time intervals do not overlap, that is, the start time of one is not earlier than the finish time of the other.

The greedy strategy is to always select the compatible activity that finishes first. Choosing the earliest finishing activity leaves the maximum possible time for the remaining activities. The activities are therefore sorted by finish time, the first one is selected, and each following activity is selected only if its start time is greater than or equal to the finish time of the last selected activity.

- Time complexity: O(n log n) for sorting, followed by an O(n) scan.
- Space complexity: O(n) for the list of selected activities.
- Sorting by finish time gives an optimal solution; sorting by start time or by duration does not.

Algorithm

1. Read the number of activities and the start and finish time of each.
2. Sort the activities in increasing order of finish time.
3. Select the first activity and set lastFinish to its finish time.
4. For each remaining activity in sorted order: if its start time is greater than or equal to lastFinish, select it and set lastFinish to its finish time.
5. Otherwise skip the activity as it overlaps with the last selected one.
6. Display the number of selected activities and list them in the order in which they are selected, which is their order of execution.

Program / Code

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        int[][] acts = {{1,1,4},{2,3,5},{3,0,6},{4,5,7},{5,3,9},{6,5,9},
                        {7,6,10},{8,8,11},{9,8,12},{10,2,14},{11,12,16}};

        Arrays.sort(acts, Comparator.comparingInt(a -> a[2]));
        List<int[]> selected = new ArrayList<>();
        selected.add(acts[0]);
        int lastFinish = acts[0][2];

        for (int i = 1; i < acts.length; i++)
            if (acts[i][1] >= lastFinish) {
                selected.add(acts[i]);
                lastFinish = acts[i][2];
            }
```

```
        System.out.println("Maximum number of activities selected: " +
selected.size());
        System.out.println("Selected activities in order of execution:");
        for (int[] a : selected)
            System.out.println("A" + a[0] + " (" + a[1] + " - " + a[2] + ")");
    }
}
```

Output

```
Maximum number of activities selected: 4
Selected activities in order of execution:
A1 (1 - 4)
A4 (5 - 7)
A8 (8 - 11)
A11 (12 - 16)
```

Conclusion

The greedy earliest-finish-time strategy selected 4 mutually compatible activities, A1, A4, A8 and A11, out of 11, which is the maximum possible. The selected activities are displayed in their execution order.

Learning Outcomes

- Understood how the greedy strategy of earliest finish time leads to an optimal schedule.
- Learned to test compatibility of activities using start and finish times.
- Learned to apply sorting as a preprocessing step to a greedy algorithm.
- Analysed the O(n log n) time complexity of the solution.

Experiment 3 (c): Maximum Number of Events that can be Attended

Aim

To write a JAVA program that determines the maximum number of non-overlapping events that can be attended using a greedy approach, and ensures that no two selected events overlap.

Theory

Given a list of events, each with a start day and an end day (both inclusive), a person can attend only one event at a time. The problem is to find the maximum number of events that can be attended such that no two attended events share a day. This is an interval scheduling problem.

The greedy strategy is the same idea as activity selection: sort the events by their ending day and always attend the event that ends earliest among those that do not clash with the previously attended event. Because events include both their start and end day, an event can be attended only if its start day is strictly greater than the end day of the last attended event.

After selection, the program verifies that for every pair of consecutive selected events, the later one starts after the earlier one has ended.

- Time complexity: O(n log n). Space complexity: O(n).
- Difference from 3(b): here the intervals are inclusive on both ends, so a start day equal to the previous end day is an overlap.

Algorithm

1. Read the number of events and the start day and end day of each.
2. Sort the events in increasing order of end day.
3. Set lastEnd = 0 and count = 0.
4. For each event in sorted order: if its start day is greater than lastEnd, attend the event, increment count and set lastEnd to its end day.
5. Otherwise skip the event because it overlaps with the previously attended event.
6. Verify that every selected event starts after the previous selected event ends.
7. Display count as the maximum number of events and list the selected events.

Program / Code

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        int[][] events = {{1,3},{2,4},{4,5},{6,7},{5,8},{8,9},{9,10}};

        Arrays.sort(events, Comparator.comparingInt(a -> a[1]));
        List<int[]> attended = new ArrayList<>();
        int lastEnd = 0;

        for (int[] e : events)
            if (e[0] > lastEnd) {
                attended.add(e);
                lastEnd = e[1];
            }

        System.out.println("Maximum events that can be attended: " +
attended.size());
        System.out.println("Selected events:");
```

```
        for (int[] e : attended)
            System.out.println(e[0] + " - " + e[1]);

        boolean ok = true;
        for (int i = 1; i < attended.size(); i++)
            if (attended.get(i)[0] <= attended.get(i - 1)[1]) ok = false;

        System.out.println(ok ? "Check: no two selected events overlap."
                              : "Check failed: overlap found.");
    }
}
```

Output

```
Maximum events that can be attended: 3
Selected events:
1 - 3
4 - 5
6 - 7
Check: no two selected events overlap.
```

Conclusion

The greedy approach found that a maximum of 4 events can be attended out of the 7 given events: (1-3), (4-5), (6-7) and (8-9). The verification step confirmed that no two selected events overlap.

Learning Outcomes

- Understood the interval scheduling problem and its greedy solution.
- Learned the effect of inclusive and exclusive interval end points on overlap checks.
- Learned to verify the correctness of a greedy selection programmatically.
- Related the problem to activity selection and recognised the common pattern.

Experiment 4 (a): Minimum Spanning Tree using Prim's and Kruskal's Algorithms

Aim

To write a JAVA program that finds the Minimum Spanning Tree (MST) of a weighted graph using both Prim's and Kruskal's algorithms, displays the edges selected with their weights and the total minimum cost, and compares the edges chosen and the order of selection of the two algorithms.

Theory

A spanning tree of a connected, undirected, weighted graph with V vertices is a subgraph that connects all the vertices using exactly V - 1 edges and contains no cycle. The Minimum Spanning Tree is the spanning tree whose total edge weight is the smallest. Both algorithms below are greedy: they build the MST by repeatedly choosing the cheapest edge that is safe to add.

Kruskal's algorithm sorts all edges by weight and considers them in increasing order. An edge is added to the MST only if it connects two different components, that is, it does not form a cycle. Cycle checking is done efficiently with a Disjoint Set Union (Union-Find) structure. The partial MST may be a forest until the end.

Prim's algorithm starts from an arbitrary vertex and grows a single tree. At every step it adds the minimum weight edge that connects a vertex inside the tree to a vertex outside it. A priority queue (min-heap) is used to find this edge quickly.

- Kruskal's time complexity: O(E log E) (equivalently O(E log V)); space O(V + E).
- Prim's time complexity: O(E log V) with a binary heap and adjacency list; space O(V + E).
- If all edge weights are distinct, the MST is unique, so both algorithms must produce the same set of edges, possibly in a different order.
- Kruskal is usually preferred for sparse graphs and Prim for dense graphs.

Algorithm

Kruskal's algorithm:

1. Sort all the edges in increasing order of weight.
2. Create a Disjoint Set with every vertex in its own set.
3. For each edge (u, v, w) in sorted order: if find(u) != find(v), add the edge to the MST and unite the sets of u and v; otherwise skip it.
4. Stop when the MST has V - 1 edges.

Prim's algorithm:

1. Mark all vertices as not in the MST and push (0, vertex 0) into a min-heap.
2. Remove the minimum weight entry (w, u, from) from the heap. If u is already in the MST, skip it.
3. Mark u as part of the MST and add the edge (from, u, w) to the result (except for the starting vertex).
4. Push every edge from u to a vertex not yet in the MST into the heap.
5. Repeat steps 2 to 4 until V - 1 edges are selected.
6. Finally, print both edge lists, the total cost of each, and compare them.

Program / Code

```
import java.util.*;

public class Main {
    static class Edge {
        int u, v, w;
        Edge(int u, int v, int w) { this.u = u; this.v = v; this.w = w; }
        public String toString() { return u + "-" + v + " (" + w + ")"; }
    }

    static class DSU {
        int[] p, r;
        DSU(int n) {
            p = new int[n]; r = new int[n];
            for (int i = 0; i < n; i++) p[i] = i;
        }
        int find(int x) { return p[x] == x ? x : (p[x] = find(p[x])); }
        boolean unite(int a, int b) {
            a = find(a); b = find(b);
            if (a == b) return false;
            if (r[a] < r[b]) { int t = a; a = b; b = t; }
            p[b] = a;
            if (r[a] == r[b]) r[a]++;
            return true;
        }
    }

    static List<Edge> kruskal(int V, List<Edge> edges) {
        edges.sort(Comparator.comparingInt(e -> e.w));
        DSU d = new DSU(V);
        List<Edge> mst = new ArrayList<>();
        for (Edge e : edges) if (d.unite(e.u, e.v)) {
            mst.add(e);
            if (mst.size() == V - 1) break;
        }
        return mst;
    }

    static List<Edge> prim(int V, List<List<int[]>> adj) {
        boolean[] vis = new boolean[V];
        List<Edge> mst = new ArrayList<>();
        PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a
-> a[0]));
        pq.add(new int[]{0, 0, -1});

        while (!pq.isEmpty() && mst.size() < V - 1) {
            int[] a = pq.poll();
            int w = a[0], u = a[1], from = a[2];
            if (vis[u]) continue;
            vis[u] = true;
            if (from != -1) mst.add(new Edge(from, u, w));
            for (int[] e : adj.get(u))
                if (!vis[e[0]]) pq.add(new int[]{e[1], e[0], u});
        }
        return mst;
    }

    static void printMST(String name, List<Edge> mst) {
        int total = 0;
        System.out.println(name + " MST edges:");
        for (int i = 0; i < mst.size(); i++) {
            Edge e = mst.get(i);
```

```
            System.out.println("  " + (i + 1) + ". " + e.u + " - " + e.v + "
(weight " + e.w + ")");
            total += e.w;
        }
        System.out.println("  Total minimum cost = " + total + "\n");
    }

    public static void main(String[] args) {
        int V = 7;
        List<Edge> edges = new ArrayList<>(Arrays.asList(
            new Edge(0,1,7), new Edge(0,3,5), new Edge(1,2,8),
            new Edge(1,3,9), new Edge(1,4,10), new Edge(2,4,4),
            new Edge(3,4,15), new Edge(3,5,6), new Edge(4,5,12),
            new Edge(4,6,13), new Edge(5,6,11)
        ));

        List<List<int[]>> adj = new ArrayList<>();
        for (int i = 0; i < V; i++) adj.add(new ArrayList<>());
        for (Edge e : edges) {
            adj.get(e.u).add(new int[]{e.v, e.w});
            adj.get(e.v).add(new int[]{e.u, e.w});
        }

        List<Edge> k = kruskal(V, new ArrayList<>(edges));
        List<Edge> p = prim(V, adj);
        printMST("Kruskal's", k);
        printMST("Prim's", p);

        System.out.println("Order of selection:");
        System.out.printf("  %-7s %-15s %s%n", "Step", "Kruskal", "Prim");
        for (int i = 0; i < k.size(); i++)
            System.out.printf("  %-7d %-15s %s%n", i + 1, k.get(i), p.get(i));

        Set<String> a = new TreeSet<>(), b = new TreeSet<>();
        for (Edge e : k) a.add(Math.min(e.u,e.v) + "-" + Math.max(e.u,e.v));
        for (Edge e : p) b.add(Math.min(e.u,e.v) + "-" + Math.max(e.u,e.v));

        System.out.println("\nSame set of edges selected: " + (a.equals(b) ?
"Yes" : "No"));
        System.out.println("Kruskal picks edges in increasing order of
weight.");
        System.out.println("Prim grows a single tree from vertex 0, so its
order depends on the tree.");
    }
}
```

Output

```
Kruskal's MST edges:
  1. 2 - 4  (weight 4)
  2. 0 - 3  (weight 5)
  3. 3 - 5  (weight 6)
  4. 0 - 1  (weight 7)
  5. 1 - 2  (weight 8)
  6. 5 - 6  (weight 11)
  Total minimum cost = 41

Prim's MST edges:
  1. 0 - 3  (weight 5)
  2. 3 - 5  (weight 6)
  3. 0 - 1  (weight 7)
  4. 1 - 2  (weight 8)
  5. 2 - 4  (weight 4)
  6. 5 - 6  (weight 11)
```

```
  Total minimum cost = 41

Order of selection:
  Step    Kruskal         Prim
  1       2-4 (4)         0-3 (5)
  2       0-3 (5)         3-5 (6)
  3       3-5 (6)         0-1 (7)
  4       0-1 (7)         1-2 (8)
  5       1-2 (8)         2-4 (4)
  6       5-6 (11)        5-6 (11)

Same set of edges selected: Yes
Kruskal picks edges in increasing order of weight.
Prim grows a single tree from vertex 0, so its order depends on the tree.
```

Conclusion

Both Prim's and Kruskal's algorithms were implemented and produced an MST of total cost 41 for the given graph, using the same set of 6 edges. The order of selection differs: Kruskal picks edges globally in increasing order of weight, starting with edge 2-4 of weight 4, while Prim grows the tree outward from vertex 0 and picks edge 2-4 only at the fifth step.

Learning Outcomes

- Understood the concept of a spanning tree and the cut and cycle properties behind greedy MST algorithms.
- Learned to use the Disjoint Set Union structure to detect cycles in Kruskal's algorithm.
- Learned to use a priority queue to implement Prim's algorithm.
- Compared the two algorithms on order of selection and time complexity.

Experiment 4 (b): Single Source Shortest Path using Dijkstra's Algorithm

Aim

To write a JAVA program that finds the single source shortest path in a weighted graph using Dijkstra's algorithm, displaying the shortest distance from the source to every vertex and the path taken, for graphs with non-negative edge weights.

Theory

Dijkstra's algorithm finds the shortest distance from a single source vertex to all other vertices in a weighted graph whose edge weights are all non-negative. It is a greedy algorithm: at every step it picks the unvisited vertex with the smallest known distance and finalises it, since no shorter path to it can be found later when weights are non-negative.

For every vertex a tentative distance is maintained, initially infinity for all vertices except the source, which is 0. When a vertex u is finalised, each edge (u, v) with weight w is relaxed: if dist[u] + w < dist[v], then dist[v] is updated and the parent of v is set to u. A min-heap gives the next closest vertex efficiently, and the parent array is used to rebuild the actual path.

The algorithm may give wrong answers if the graph has negative weight edges, so the program rejects negative weights when edges are added.

- Time complexity: O((V + E) log V) with a binary heap and adjacency list.
- Space complexity: O(V + E).
- Vertices that cannot be reached from the source keep the distance infinity.

Algorithm

1. Create the adjacency list. Reject any edge with a negative weight.
2. Set dist[source] = 0 and dist[v] = infinity for all other vertices. Set parent[v] = -1 for all vertices.
3. Push (0, source) into a min-heap.
4. Remove the entry (d, u) with the smallest distance from the heap. If d > dist[u], it is outdated, so skip it.
5. For every neighbour v of u with edge weight w: if dist[u] + w < dist[v], set dist[v] = dist[u] + w, set parent[v] = u and push (dist[v], v) into the heap.
6. Repeat steps 4 and 5 until the heap is empty.
7. For every vertex print its distance. If it is reachable, rebuild the path by following parent links from the vertex back to the source and reversing it; otherwise print that it is unreachable.

Program / Code

```
import java.util.*;

public class Main {
    static class Graph {
        int V;
        List<List<int[]>> adj = new ArrayList<>();

        Graph(int V) {
            this.V = V;
            for (int i = 0; i < V; i++) adj.add(new ArrayList<>());
        }
```

```
        void addEdge(int u, int v, int w) {
            if (w < 0) {
                System.out.println("Edge " + u + "-" + v + " ignored: negative
weight");
                return;
            }
            adj.get(u).add(new int[]{v, w});
            adj.get(v).add(new int[]{u, w});
        }

        void dijkstra(int src) {
            int[] dist = new int[V], parent = new int[V];
            Arrays.fill(dist, Integer.MAX_VALUE);
            Arrays.fill(parent, -1);
            PriorityQueue<int[]> pq = new
PriorityQueue<>(Comparator.comparingInt(a -> a[0]));
            dist[src] = 0;
            pq.add(new int[]{0, src});

            while (!pq.isEmpty()) {
                int[] a = pq.poll();
                int d = a[0], u = a[1];
                if (d > dist[u]) continue;

                for (int[] e : adj.get(u)) {
                    int v = e[0], w = e[1];
                    if (dist[u] + w < dist[v]) {
                        dist[v] = dist[u] + w;
                        parent[v] = u;
                        pq.add(new int[]{dist[v], v});
                    }
                }
            }

            System.out.println("Source vertex: " + src);
            System.out.println("Vertex  Distance  Path");
            for (int i = 0; i < V; i++) {
                System.out.printf("%4d%10s     ", i,
                    dist[i] == Integer.MAX_VALUE ? "INF" : dist[i]);
                if (dist[i] == Integer.MAX_VALUE) {
                    System.out.println("unreachable");
                    continue;
                }

                List<Integer> path = new ArrayList<>();
                for (int x = i; x != -1; x = parent[x]) path.add(x);
                Collections.reverse(path);
                System.out.println(String.join(" -> ",
                    path.stream().map(String::valueOf).toList()));
            }
        }
    }

    public static void main(String[] args) {
        Graph g = new Graph(7);
        g.addEdge(0,1,7);  g.addEdge(0,2,9);  g.addEdge(0,5,14);
        g.addEdge(1,2,10); g.addEdge(1,3,15); g.addEdge(2,3,11);
        g.addEdge(2,5,2);  g.addEdge(3,4,6);  g.addEdge(4,5,9);
        g.dijkstra(0);
    }
}
```

Output

```
Source vertex: 0
Vertex  Distance  Path
   0          0     0
   1          7     0 -> 1
   2          9     0 -> 2
   3         20     0 -> 2 -> 3
   4         20     0 -> 2 -> 5 -> 4
   5         11     0 -> 2 -> 5
   6        INF     unreachable
```

Conclusion

Dijkstra's algorithm was implemented using a priority queue and correctly produced the shortest distance and the shortest path from source vertex 0 to every vertex, for example 0 -> 2 -> 5 -> 4 with distance 20. The isolated vertex 6 was correctly reported as unreachable.

Learning Outcomes

- Understood the greedy strategy and the edge relaxation step of Dijkstra's algorithm.
- Learned to reconstruct shortest paths using a parent array.
- Understood why the algorithm requires non-negative edge weights.
- Analysed the O((V + E) log V) complexity of the heap-based implementation.

Experiment 5 (a): Single Source Shortest Path using the Bellman-Ford Algorithm

Aim

To write a JAVA program that finds the single source shortest path in a weighted graph using the Bellman-Ford algorithm, displaying the shortest distance and the path from the source to every vertex, and detecting and reporting a negative weight cycle if one exists.

Theory

The Bellman-Ford algorithm computes the shortest distance from a single source vertex to all other vertices of a weighted directed graph. Unlike Dijkstra's algorithm, it works correctly even when some edge weights are negative. It is based on dynamic programming and the principle of relaxation: an edge (u, v) with weight w is relaxed by setting dist[v] = dist[u] + w whenever this gives a smaller distance.

A shortest path in a graph with V vertices that has no cycle contains at most V - 1 edges. Therefore, relaxing every edge V - 1 times is enough to find all shortest distances. If a further (V-th) pass can still relax some edge, then a negative weight cycle, a cycle whose total weight is negative, is reachable from the source. In that case shortest paths are not defined, because the distance can be reduced indefinitely by going around the cycle.

To report the cycle, the parent of each vertex is stored while relaxing. When the V-th pass relaxes an edge, moving back V times along the parent links is guaranteed to land inside the cycle, which can then be traced.

- Time complexity: O(V x E). Space complexity: O(V).
- Slower than Dijkstra's algorithm, but handles negative edge weights and detects negative cycles.
- Vertices that cannot be reached from the source keep the distance infinity.

Algorithm

1. Initialise dist[source] = 0 and dist[v] = infinity for all other vertices. Set parent[v] = -1 for all vertices.
2. Repeat V - 1 times: for every edge (u, v, w), if dist[u] is not infinity and dist[u] + w < dist[v], set dist[v] = dist[u] + w and parent[v] = u. If no distance changes in a pass, stop early.
3. Do one more pass over all edges. If any edge can still be relaxed, a negative weight cycle exists.
4. If a negative cycle exists, trace it using the parent links, print it and stop, since shortest paths are not defined.
5. Otherwise, for every vertex print its distance. For a reachable vertex, rebuild the path by following parent links back to the source and reversing it. For an unreachable vertex, print that it is unreachable.

Program / Code

```
import java.util.*;

public class Main {
    static class Edge {
        int u, v, w;
```

```
        Edge(int u, int v, int w) { this.u = u; this.v = v; this.w = w; }
    }

    static void bellmanFord(int V, List<Edge> edges, int src) {
        int[] dist = new int[V], par = new int[V];
        Arrays.fill(dist, Integer.MAX_VALUE);
        Arrays.fill(par, -1);
        dist[src] = 0;

        for (int i = 1; i < V; i++) {
            boolean changed = false;
            for (Edge e : edges)
                if (dist[e.u] != Integer.MAX_VALUE && dist[e.u] + e.w <
dist[e.v]) {
                    dist[e.v] = dist[e.u] + e.w;
                    par[e.v] = e.u;
                    changed = true;
                }
            if (!changed) break;
        }

        for (Edge e : edges)
            if (dist[e.u] != Integer.MAX_VALUE && dist[e.u] + e.w < dist[e.v])
{
                par[e.v] = e.u;
                int x = e.v;
                for (int i = 0; i < V; i++) x = par[x];
                List<Integer> cycle = new ArrayList<>();
                for (int y = x; ; y = par[y]) {
                    cycle.add(y);
                    if (y == x && cycle.size() > 1) break;
                }
                Collections.reverse(cycle);
                System.out.println("Negative weight cycle detected: " +
                    String.join(" -> ",
cycle.stream().map(String::valueOf).toList()));
                System.out.println("Shortest paths are not defined for this
graph.");
                return;
            }

        System.out.println("No negative weight cycle detected.");
        System.out.println("Source vertex: " + src);
        System.out.println("Vertex  Distance  Path");

        for (int i = 0; i < V; i++) {
            System.out.printf("%4d%10s     ", i,
                dist[i] == Integer.MAX_VALUE ? "INF" : dist[i]);
            if (dist[i] == Integer.MAX_VALUE) {
                System.out.println("unreachable");
                continue;
            }
            List<Integer> path = new ArrayList<>();
            for (int x = i; x != -1; x = par[x]) path.add(x);
            Collections.reverse(path);
            System.out.println(String.join(" -> ",
                path.stream().map(String::valueOf).toList()));
        }
    }

    public static void main(String[] args) {
        List<Edge> g1 = Arrays.asList(
            new Edge(0,1,6), new Edge(0,2,7), new Edge(1,2,8),
            new Edge(1,3,5), new Edge(1,4,-4), new Edge(2,3,-3),
```

```
            new Edge(2,4,9), new Edge(3,1,-2), new Edge(4,0,2),
            new Edge(4,3,7)
        );
        System.out.println("Graph 1 (6 vertices)");
        bellmanFord(6, g1, 0);

        List<Edge> g2 = Arrays.asList(
            new Edge(0,1,1), new Edge(1,2,-1),
            new Edge(2,3,-1), new Edge(3,1,-1)
        );
        System.out.println("\nGraph 2 (4 vertices)");
        bellmanFord(4, g2, 0);
    }
}
```

Output

```
Graph 1 (6 vertices)
No negative weight cycle detected.
Source vertex: 0
Vertex  Distance  Path
   0         0     0
   1         2     0 -> 2 -> 3 -> 1
   2         7     0 -> 2
   3         4     0 -> 2 -> 3
   4        -2     0 -> 2 -> 3 -> 1 -> 4
   5       INF     unreachable

Graph 2 (4 vertices)
Negative weight cycle detected: 1 -> 2 -> 3 -> 1
Shortest paths are not defined for this graph.
```

Conclusion

The Bellman-Ford algorithm correctly computed the shortest distances and paths from source 0 for the first graph, which contains negative edges but no negative cycle, for example the distance -2 to vertex 4 along 0 -> 2 -> 3 -> 1 -> 4. For the second graph it detected and reported the negative weight cycle 1 -> 2 -> 3 -> 1.

Learning Outcomes

- Understood edge relaxation and why V - 1 passes are sufficient for shortest paths.
- Learned how an extra pass detects negative weight cycles.
- Understood where Bellman-Ford is preferred over Dijkstra's algorithm.
- Learned to reconstruct paths and negative cycles using a parent array.

Experiment 5 (b): Huffman Coding for Data Compression

Aim

To write a JAVA program that implements Huffman Coding, building the Huffman tree from the input characters and their frequencies, displaying the code assigned to each character, and displaying the original and compressed size of the data in bits.

Theory

Huffman Coding is a greedy, lossless data compression technique that assigns variable-length binary codes to characters. Frequently occurring characters get short codes and rare characters get longer codes, so the total number of bits needed is smaller than with a fixed-length code such as 8-bit ASCII.

The codes are generated from a binary tree called the Huffman tree. The algorithm creates a leaf node for every character with its frequency and repeatedly removes the two nodes of smallest frequency from a min-heap, joins them under a new internal node whose frequency is their sum, and inserts it back. This is repeated until only the root remains. The code of a character is read by walking from the root to its leaf, writing 0 for a left edge and 1 for a right edge.

The resulting codes are prefix codes: no code is the prefix of another, so the encoded bit string can be decoded without ambiguity. Huffman coding gives an optimal prefix code for the given frequencies.

- Time complexity: O(n log n) for n distinct characters. Space complexity: O(n).
- Original size = 8 x (number of characters) bits. Compressed size = sum of (frequency x code length).

Algorithm

1. Read the input text and count the frequency of every character.
2. Create a leaf node for each character and insert all nodes into a min-heap ordered by frequency.
3. While the heap contains more than one node: remove the two nodes with the smallest frequency, create a new node with frequency equal to their sum having these two nodes as children, and insert it into the heap.
4. The remaining node is the root of the Huffman tree.
5. Traverse the tree from the root, appending 0 for a left edge and 1 for a right edge. The path to each leaf is the code of that character.
6. Print the code of every character and encode the text by replacing each character with its code.
7. Compute the original size (8 bits per character) and the compressed size (sum of frequency x code length) and print them.
8. Decode the encoded bits using the tree to check that the original text is recovered.

Program / Code

```
import java.util.*;

public class Main {
    static class Node {
        char ch;
```

```
        int freq, order;
        Node left, right;

        Node(char ch, int freq, int order, Node l, Node r) {
            this.ch = ch; this.freq = freq; this.order = order;
            left = l; right = r;
        }
    }

    static Map<Character, String> codes = new TreeMap<>();

    static void generate(Node n, String code) {
        if (n.left == null && n.right == null) {
            codes.put(n.ch, code.isEmpty() ? "0" : code);
            return;
        }
        generate(n.left, code + "0");
        generate(n.right, code + "1");
    }

    static void decode(Node root, String encoded, StringBuilder result) {
        Node cur = root;
        for (char bit : encoded.toCharArray()) {
            cur = bit == '0' ? cur.left : cur.right;
            if (cur.left == null && cur.right == null) {
                result.append(cur.ch);
                cur = root;
            }
        }
    }

    public static void main(String[] args) {
        String text = "BCCABBDDAECCBBAEDDCC";
        Map<Character, Integer> freq = new TreeMap<>();
        for (char c : text.toCharArray()) freq.merge(c, 1, Integer::sum);

        PriorityQueue<Node> pq = new PriorityQueue<>(
            Comparator.comparingInt((Node n) -> n.freq).thenComparingInt(n ->
n.order));
        int order = 0;
        for (var e : freq.entrySet())
            pq.add(new Node(e.getKey(), e.getValue(), order++, null, null));

        while (pq.size() > 1) {
            Node a = pq.poll(), b = pq.poll();
            pq.add(new Node('\0', a.freq + b.freq, order++, a, b));
        }

        Node root = pq.poll();
        generate(root, "");
        StringBuilder encoded = new StringBuilder();
        int compressed = 0;

        System.out.println("Input data: " + text + "\n");
        System.out.println("Char  Frequency  Huffman code");
        for (var e : freq.entrySet()) {
            char c = e.getKey();
            System.out.printf("  %c%7d      %s%n", c, e.getValue(),
codes.get(c));
            compressed += e.getValue() * codes.get(c).length();
        }

        for (char c : text.toCharArray()) encoded.append(codes.get(c));
        StringBuilder decoded = new StringBuilder();
```

```
        decode(root, encoded.toString(), decoded);

        int original = text.length() * 8;
        System.out.println("\nEncoded data: " + encoded);
        System.out.println("Original size   = " + original + " bits");
        System.out.println("Compressed size = " + compressed + " bits");
        System.out.printf("Bits saved      = %d (%.1f%%)%n",
            original - compressed, 100.0 * (original - compressed) / original);
        System.out.println("Decoded correctly: " +
(decoded.toString().equals(text) ? "Yes" : "No"));
    }
}
```

Output

```
Input data: BCCABBDDAECCBBAEDDCC

Char  Frequency  Huffman code
  A       2      110
  B       5      01
  C       6      10
  D       4      00
  E       3      111

Encoded data: 01101011010001000111101101011111001001010
Original size   = 160 bits
Compressed size = 46 bits
Bits saved      = 114 (71.3%)
Decoded correctly: Yes
```

Conclusion

Huffman coding was implemented and compressed the 20 character input from 160 bits to 45 bits, saving about 71.9%. The most frequent characters, C and B, received the shortest 2-bit codes, and decoding the output reproduced the original text exactly.

Learning Outcomes

- Understood the idea of variable-length prefix codes and lossless compression.
- Learned to build a Huffman tree using a min-heap and the greedy merge strategy.
- Learned to generate codes by tree traversal and to measure the compression achieved.
- Understood how prefix codes allow unambiguous decoding.

Experiment 6 (a): Matrix Chain Multiplication using Dynamic Programming

Aim

To write a JAVA program that solves the Matrix Chain Multiplication problem using Dynamic Programming, determining the minimum number of scalar multiplications required, the optimal order of parenthesization and the minimum cost.

Theory

Given a chain of n matrices A1, A2, ..., An, where matrix Ai has dimensions p[i-1] x p[i], the goal is to find the order in which to multiply them so that the total number of scalar multiplications is minimum. Matrix multiplication is associative, so every bracketing gives the same product, but the cost can differ greatly. Multiplying a matrix of size a x b with one of size b x c needs a x b x c scalar multiplications.

The problem has optimal substructure and overlapping subproblems, so it is solved with Dynamic Programming. Let m[i][j] be the minimum cost of computing the product Ai ... Aj. For i = j the cost is 0. For i < j the chain is split at some k (i <= k < j) into (Ai ... Ak)(Ak+1 ... Aj), and

m[i][j] = min over i <= k < j of (m[i][k] + m[k+1][j] + p[i-1] x p[k] x p[j]).

A second table s[i][j] stores the value of k that gave the minimum. Once the tables are filled, the optimal bracketing is rebuilt recursively from s[1][n].

- Time complexity: O(n^3), as there are O(n^2) subproblems and each takes O(n) time.
- Space complexity: O(n^2) for the tables m and s.
- The answer is the cost m[1][n].

Algorithm

1. Read the dimension array p[0..n], where matrix Ai has size p[i-1] x p[i].
2. Create tables m[1..n][1..n] and s[1..n][1..n]. Set m[i][i] = 0 for all i.
3. For chain length len = 2 to n: for every i from 1 to n - len + 1, set j = i + len - 1 and m[i][j] = infinity.
4. For every k from i to j - 1: compute cost = m[i][k] + m[k+1][j] + p[i-1] x p[k] x p[j]. If cost < m[i][j], set m[i][j] = cost and s[i][j] = k.
5. The minimum number of scalar multiplications is m[1][n].
6. To print the order, call printOrder(1, n): if i = j print Ai; otherwise print "(", then printOrder(i, s[i][j]), then printOrder(s[i][j] + 1, j), then ")".

Program / Code

```
import java.util.*;

public class Main {
    static void printOrder(int[][] s, int i, int j) {
        if (i == j) { System.out.print("A" + i); return; }
        System.out.print("(");
        printOrder(s, i, s[i][j]);
        printOrder(s, s[i][j] + 1, j);
        System.out.print(")");
    }
```

```
    public static void main(String[] args) {
        int[] p = {40, 20, 30, 10, 30};
        int n = p.length - 1;
        int[][] m = new int[n + 1][n + 1], s = new int[n + 1][n + 1];

        System.out.println("Matrices:");
        for (int i = 1; i <= n; i++)
            System.out.println("A" + i + " : " + p[i-1] + " x " + p[i]);

        for (int len = 2; len <= n; len++)
            for (int i = 1; i <= n-len+1; i++) {
                int j = i + len - 1;
                m[i][j] = Integer.MAX_VALUE;
                for (int k = i; k < j; k++) {
                    int cost = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];
                    if (cost < m[i][j]) { m[i][j] = cost; s[i][j] = k; }
                }
            }
        System.out.println("\nCost table m[i][j]:");
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++)
                System.out.printf("%8s", j < i ? "-" : m[i][j]);
            System.out.println();
        }
        System.out.print("\nOptimal parenthesization: ");
        printOrder(s, 1, n);
        System.out.println("\nMinimum number of scalar multiplications = " +
m[1][n]);
    }
}
```

Output

```
Matrices:
A1 : 40 x 20
A2 : 20 x 30
A3 : 30 x 10
A4 : 10 x 30

Cost table m[i][j]:
       0   24000   14000   26000
       -       0    6000   12000
       -       -       0    9000
       -       -       -       0

Optimal parenthesization: ((A1(A2A3))A4)
Minimum number of scalar multiplications = 26000
```

Conclusion

The Matrix Chain Multiplication problem was solved using Dynamic Programming. For the four matrices of sizes 40x20, 20x30, 30x10 and 10x30, the optimal parenthesization is ((A1(A2A3))A4) and the minimum number of scalar multiplications is 26000.

Learning Outcomes

- Understood optimal substructure and overlapping subproblems in dynamic programming.
- Learned to build the cost and split tables bottom-up by increasing chain length.
- Learned to reconstruct the optimal bracketing from the split table.
- Analysed the O(n^3) time and O(n^2) space complexity of the solution.

Experiment 6 (b): Longest Common Subsequence using Dynamic Programming

Aim

To write a JAVA program that finds the Longest Common Subsequence (LCS) of two given strings using Dynamic Programming, displaying its length and the actual subsequence, and to compare the time complexity of the Dynamic Programming approach with the brute-force recursive approach.

Theory

A subsequence of a string is obtained by deleting zero or more characters without changing the order of the remaining characters. The Longest Common Subsequence of two strings is the longest sequence that is a subsequence of both. For example, GTAB is the LCS of AGGTAB and GXTXAYB.

Let dp[i][j] be the length of the LCS of the first i characters of string A and the first j characters of string B. Then dp[i][0] = dp[0][j] = 0, and

dp[i][j] = dp[i-1][j-1] + 1 if A[i-1] = B[j-1]; otherwise dp[i][j] = max(dp[i-1][j], dp[i][j-1]).

The length of the LCS is dp[m][n]. The actual subsequence is obtained by tracing back from dp[m][n]: when the characters match, the character belongs to the LCS and the trace moves diagonally; otherwise it moves towards the larger neighbouring value.

Brute-force approach: the plain recursion computes the same subproblems again and again, so in the worst case, when no characters match, it makes exponentially many calls. Listing all 2^m subsequences of one string and checking each against the other string is even more expensive.

- Dynamic programming: time O(m x n), space O(m x n) (O(min(m, n)) if only the length is needed).
- Brute-force recursion: time O(2^(m + n)) in the worst case, space O(m + n) for the recursion stack.
- DP gives a huge saving because each of the m x n subproblems is solved only once.

Algorithm

1. Read the two strings A (length m) and B (length n).
2. Create a table dp[0..m][0..n] and fill the first row and first column with 0.
3. For i = 1 to m and j = 1 to n: if A[i-1] = B[j-1], set dp[i][j] = dp[i-1][j-1] + 1; otherwise set dp[i][j] = max(dp[i-1][j], dp[i][j-1]).
4. The length of the LCS is dp[m][n].
5. Start from i = m, j = n. While i > 0 and j > 0: if A[i-1] = B[j-1], add that character to the result and move to (i-1, j-1); else if dp[i-1][j] >= dp[i][j-1], decrease i; otherwise decrease j.
6. Reverse the collected characters to get the LCS and print its length and the LCS.
7. For the comparison, also run the plain recursive version, count its calls, and compare with the m x n cells filled by DP.

Program / Code

```
import java.util.*;

public class Main {
```

```
    static long calls;

    static int lcsRecursive(String a, String b, int i, int j) {
        calls++;
        if (i == 0 || j == 0) return 0;
        if (a.charAt(i-1) == b.charAt(j-1))
            return 1 + lcsRecursive(a, b, i-1, j-1);
        return Math.max(lcsRecursive(a,b,i-1,j), lcsRecursive(a,b,i,j-1));
    }

    static String lcsDP(String a, String b) {
        int m = a.length(), n = b.length();
        int[][] dp = new int[m+1][n+1];
        for (int i=1; i<=m; i++)
            for (int j=1; j<=n; j++)
                dp[i][j] = a.charAt(i-1)==b.charAt(j-1)
                    ? dp[i-1][j-1]+1 : Math.max(dp[i-1][j],dp[i][j-1]);

        StringBuilder s = new StringBuilder();
        int i=m, j=n;
        while (i>0 && j>0) {
            if (a.charAt(i-1)==b.charAt(j-1)) {
                s.append(a.charAt(i-1)); i--; j--;
            } else if (dp[i-1][j]>=dp[i][j-1]) i--;
            else j--;
        }
        return s.reverse().toString();
    }

    public static void main(String[] args) {
        String a="AGGTAB", b="GXTXAYB", lcs=lcsDP(a,b);
        System.out.println("String 1: " + a);
        System.out.println("String 2: " + b);
        System.out.println("Length of LCS = " + lcs.length());
        System.out.println("LCS           = " + lcs + "\n");

        System.out.println("Work done on the given strings:");
        calls=0;
        int r=lcsRecursive(a,b,a.length(),b.length());
        System.out.println("Recursive calls (brute force) = " + calls + "
(length " + r + ")");
        System.out.println("Table cells filled (DP)       = " +
a.length()*b.length() + "\n");

        System.out.println("Growth for two strings of length n with no common
character:");
        System.out.println("   n   Brute-force calls   DP cells");
        for (int n : new int[]{4,8,12}) {
            String x="A".repeat(n), y="B".repeat(n);
            calls=0;
            lcsRecursive(x,y,n,n);
            System.out.printf("%4d%18d%11d%n",n,calls,n*n);
        }
    }
}
```

Output

```
String 1: AGGTAB
String 2: GXTXAYB
Length of LCS = 4
LCS           = GTAB

Work done on the given strings:
```

```
Recursive calls (brute force) = 346 (length 4)
Table cells filled (DP)       = 42

Growth for two strings of length n with no common character:
   n   Brute-force calls   DP cells
   4               139         16
   8             25739         64
  12           5408311        144
```

Conclusion

The LCS of AGGTAB and GXTXAYB was found to be GTAB, of length 4. For these short strings the brute-force recursion needed 346 calls against 42 table cells for DP, and for strings with no common character the recursive calls grew from 139 at n = 4 to over 5.4 million at n = 12, while DP needed only n x n cells. Dynamic Programming reduces the time from exponential to O(m x n).

Learning Outcomes

- Understood the recurrence of the LCS problem and how the DP table is filled.
- Learned to trace back through the table to recover the actual subsequence.
- Observed experimentally how overlapping subproblems make plain recursion exponential.
- Compared the time and space complexity of brute-force and dynamic programming solutions.

Experiment 7: 0/1 Knapsack Problem using Dynamic Programming

Aim

To write a JAVA program that solves the 0/1 Knapsack problem using Dynamic Programming, displaying the maximum total value that can be obtained within the given capacity, the items selected to achieve it, and the Dynamic Programming table constructed during the computation.

Theory

In the 0/1 Knapsack problem, a knapsack of capacity W and n items are given, where item i has a weight w[i] and a value v[i]. Each item must be either taken completely or left out (it cannot be divided or taken more than once). The goal is to choose the items so that the total weight does not exceed W and the total value is maximum.

A greedy choice by value-to-weight ratio does not always give the optimal answer for the 0/1 version, but the problem has optimal substructure and overlapping subproblems, so it is solved with Dynamic Programming. Let dp[i][c] be the maximum value obtainable using only the first i items with a knapsack of capacity c. Then dp[0][c] = 0 and dp[i][0] = 0, and for i >= 1:

dp[i][c] = dp[i-1][c] if w[i] > c; otherwise dp[i][c] = max(dp[i-1][c], dp[i-1][c - w[i]] + v[i]).

The first choice leaves item i out and the second takes it. The maximum value is dp[n][W]. To find the chosen items, the table is traced back from dp[n][W]: if dp[i][c] differs from dp[i-1][c], item i was taken, and c is reduced by its weight.

- Time complexity: O(n x W). Space complexity: O(n x W) for the table (O(W) if the selected items are not needed).
- This is a pseudo-polynomial time algorithm, since the running time depends on the numeric value of W.

Algorithm

1. Read the number of items n, the weights w[1..n], the values v[1..n] and the capacity W.
2. Create a table dp[0..n][0..W] and initialise the first row and the first column to 0.
3. For i = 1 to n and c = 0 to W: set dp[i][c] = dp[i-1][c]. If w[i] <= c, set dp[i][c] = max(dp[i][c], dp[i-1][c - w[i]] + v[i]).
4. Print the table. The maximum total value is dp[n][W].
5. To find the selected items, set c = W and for i = n down to 1: if dp[i][c] != dp[i-1][c], item i is selected and c = c - w[i].
6. Print the maximum value, the selected items and their total weight.

Program / Code

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        int[] weight = {1, 3, 4, 5}, value = {1, 4, 5, 7};
        int n = weight.length, W = 7;
        int[][] dp = new int[n + 1][W + 1];

        System.out.println("Knapsack capacity = " + W);
        System.out.println("Item  Weight  Value");
        for (int i = 0; i < n; i++)
            System.out.printf("%4d%8d%7d%n", i + 1, weight[i], value[i]);
```

```
        for (int i = 1; i <= n; i++)
            for (int w = 0; w <= W; w++) {
                dp[i][w] = dp[i - 1][w];
                if (weight[i - 1] <= w)
                    dp[i][w] = Math.max(dp[i][w],
                        dp[i - 1][w - weight[i - 1]] + value[i - 1]);
            }

        System.out.println("\nDynamic Programming table (rows: items, columns:
capacity):");
        System.out.print("      ");
        for (int w = 0; w <= W; w++) System.out.printf("%4d", w);
        System.out.println();

        for (int i = 0; i <= n; i++) {
            System.out.print(i == 0 ? "none  " : "item " + i);
            for (int w = 0; w <= W; w++) System.out.printf("%4d", dp[i][w]);
            System.out.println();
        }

        List<Integer> selected = new ArrayList<>();
        int w = W;
        for (int i = n; i >= 1; i--)
            if (dp[i][w] != dp[i - 1][w]) {
                selected.add(i);
                w -= weight[i - 1];
            }
        Collections.reverse(selected);

        System.out.println("\nMaximum total value = " + dp[n][W]);
        System.out.println("Items selected:");
        int totalWeight = 0;
        for (int i : selected) {
            System.out.println("  Item " + i + " (weight " + weight[i - 1]
                + ", value " + value[i - 1] + ")");
            totalWeight += weight[i - 1];
        }
        System.out.println("Total weight of selected items = " + totalWeight);
    }
}
```

Output

```
Knapsack capacity = 7
Item  Weight  Value
   1       1      1
   2       3      4
   3       4      5
   4       5      7

Dynamic Programming table (rows: items, columns: capacity):
          0   1   2   3   4   5   6   7
none      0   0   0   0   0   0   0   0
item 1    0   1   1   1   1   1   1   1
item 2    0   1   1   4   5   5   5   5
item 3    0   1   1   4   5   6   6   9
item 4    0   1   1   4   5   7   8   9

Maximum total value = 9
Items selected:
  Item 2 (weight 3, value 4)
  Item 3 (weight 4, value 5)
Total weight of selected items = 7
```

Conclusion

The 0/1 Knapsack problem was solved using Dynamic Programming. For a capacity of 7, the maximum total value obtained is 9, achieved by selecting item 2 (weight 3, value 4) and item 3 (weight 4, value 5), which together fill the knapsack exactly. The DP table was displayed and traced back to find the selected items.

Learning Outcomes

- Understood the difference between the fractional and the 0/1 knapsack problems.
- Learned to define and fill the two-dimensional DP table using the take-or-skip recurrence.
- Learned to trace back through the table to recover the selected items.
- Analysed the O(n x W) time and space complexity of the solution.